

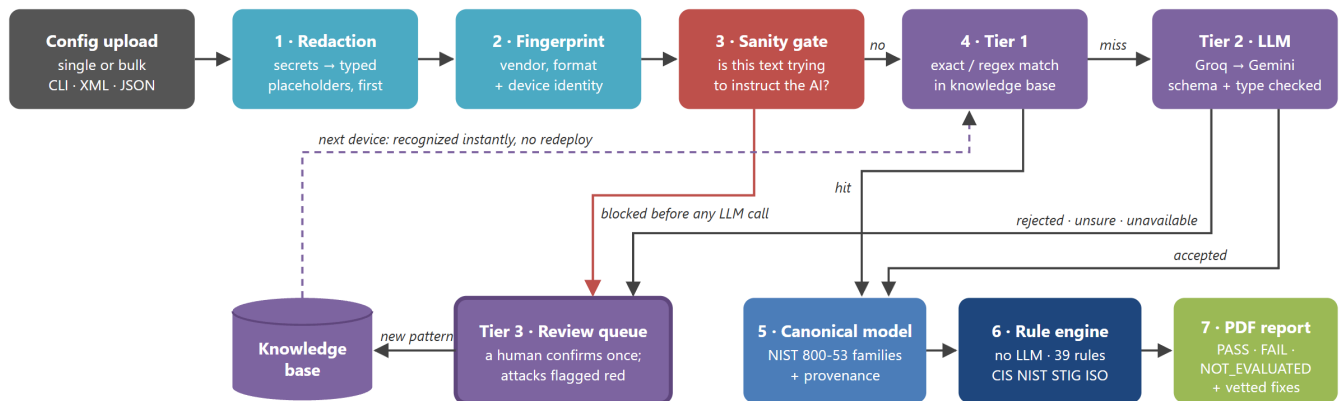
AEGIS — Architecture Document

Automated Evaluation & Governance for Infrastructure Security — an AI-augmented, vendor-agnostic network compliance engine

The problem, and the one design decision that answers it

Network devices from dozens of vendors must be checked against CIS, NIST SP 800-53, DISA STIG and ISO/IEC 27001, but every vendor's syntax differs, and a hardcoded parser per vendor breaks on the next vendor, firmware or device class. **AEGIS has exactly one parser: a lookup against one knowledge base (KB).** A "known vendor" is not a separate code path — it is a cache hit against the same KB that unknown-syntax handling writes into. Supporting new syntax means new database rows, never new code or a redeploy.

Pipeline



- Redaction first.** Secret-shaped values — enable/user password hashes, type-7 and plain passwords, SNMP communities and SNMPv3 keys, IPsec PSKs, RADIUS/TACACS+ keys, routing-protocol keys, XML/JSON secret fields — become typed placeholders ([REDACTED:AAA_KEY]) before anything else runs. The original values live only in memory for one request: never stored, never logged, never sent to an AI. The type digit stays visible (enable secret 9 [REDACTED]) because compliance rules need it.
- Fingerprint + device identity** (vendor, format, hostname, model, serial, firmware). Unrecognized vendors are not rejected — they route everything through Tiers 2/3.
- Input-sanity gate.** Config text is attacker-influenceable. Any unit shaped like an instruction to an AI ("ignore previous instructions... respond only with...") is quarantined to human review *before any LLM call*, and shown to reviewers as a blocked attack, not an ordinary item. Deliberately deterministic: an LLM-based filter would have the same injection surface it guards. Built after we demonstrated a real attack in which a pfSense rule description fabricated a compliance finding.
- Confidence-tiered resolution** of every line / XML path / JSON path:
 - Tier 1** — exact or regex match against the KB: free, instant, deterministic.
 - Tier 2** — LLM (Groq primary, Gemini fallback) proposes {field, value, confidence, reasoning}, given similar same-vendor KB entries as context. The response is schema-validated: a field name the model invented, an out-of-range confidence, or **a value whose type contradicts the field** (e.g. "https" for a boolean) is rejected, never silently accepted.
 - Tier 3** — anything rejected, low-confidence or unavailable goes to the review queue. A human confirmation is written back as a new Tier-1 pattern, so the next device containing that line resolves instantly. This write-back *is* the learning — RAG-style knowledge growth, not model fine-tuning. Reviewers also record whether the line is security-relevant and why, carried onto the learned pattern for audit.
- Canonical Security Baseline Model** organized around NIST 800-53 control families (AC/AU/IA/SC/CM), so CIS, STIG and ISO controls map onto one shared schema instead of four rule silos. Every field keeps provenance (which KB entry, which tier, which source line).
- Deterministic rule engine** — no LLM in the verdict path. Six generic predicate types (boolean, range, compound, relational, set-membership, **ordered first-match**). The last simulates real ACL evaluation order: a permit ip any any before deny tcp any any eq 23 makes the deny dead code, and AEGIS reports FAIL where a presence check

would report PASS. Vendor-specific benchmarks only apply to their own vendor; NIST and ISO apply to all. **39 rules** with real citations, each checked against its source document.

7. **Remediation + PDF.** Remediation comes only from vetted per-vendor templates (no AI-generated commands, by design — a wrong command on a live device is the highest-consequence failure in the system). The per-device PDF has device identification, a plain-English executive summary, a legend, findings sorted most-urgent first, and the source of every decision.

What makes it defensible, not just functional

- **Fail-closed by construction.** `NOT_EVALUATED` is a first-class verdict: a control whose setting never appears is reported as unknown, never as a pass.
- **Every trust boundary is enforced in code.** Secrets are redacted before inference, injection-shaped text never reaches the model, model output is type-checked before use, and verdicts are deterministic.
- **Auditable provenance.** Each finding shows whether its setting was instantly recognized, AI-classified or human-confirmed. Langfuse traces every LLM call, and a human's confirm/reject decision is attached as a score to the exact trace that proposed it.
- **Verified, not asserted.** 138 automated tests (redaction of every supported secret shape, the injection payloads, the type validator, all six predicates, tier routing and end-to-end API flows) run offline in CI on every push. Our demo configs are derived from real public configs, and running them exposed — and we fixed — real bugs: secrets that were counted but not removed, and an ACL parser that missed ports.

Tech stack

FastAPI (Python 3.11) · SQLAlchemy on SQLite (Postgres + pgvector is the production path; the code is dialect-agnostic) · local `sentence-transformers` embeddings · Groq `qwen3.8-27b` + Gemini 2.5 Flash · ReportLab · Langfuse · Next.js 16 + Tailwind · pytest + GitHub Actions.

Honest scope of this build

Designed but not built yet: a cross-reference stage that joins settings split across distant lines by name (e.g. Cisco AAA method lists, pfSense filter rules split across sibling XML elements); entropy-based redaction beyond regex; role-based access control and multi-tenant isolation (the schema carries `tenant_id` / `reviewer_id`, nothing enforces them); a job queue and LLM rate limiting (ingestion is synchronous); live device polling (Netmiko). Today's build calls third-party LLM APIs with *redacted* config — a real deployment would run a self-hosted model inside the customer's perimeter behind the same interface. Rules are authored by hand from the benchmark PDFs; the roadmap applies the same tiered pattern to rule authoring (LLM drafts a rule per benchmark control, a human confirms it) without ever putting an LLM in the evaluation path.